

Celestial Mechanics — Detailed Physics Reference

Table of Contents

- [1. Foundational Physics: The N-Body Problem](#)
- [2. Gravitational Softening](#)
- [3. Numerical Integration: Moving Bodies Through Time](#)
 - 3.1 Euler Integrator (Reference Only)
 - 3.2 Velocity Verlet (Leapfrog KDK) — **Primary**
 - 3.3 Runge-Kutta 4th Order (RK4)
 - 3.4 Yoshida 4th-Order Symplectic — **Native Engine**
- [4. Adaptive Timestep Control](#)
- [5. Barnes-Hut \$O\(N \log N\)\$ Gravity Approximation](#)
- [6. Collision Physics](#)
 - 6.1 Detection
 - 6.2 Elastic Collisions
 - 6.3 Inelastic Collisions
 - 6.4 Merge / Accretion (Conservation Laws)
- [7. Post-Newtonian Relativistic Corrections \(1PN\)](#)
- [8. Gravitational Wave Physics](#)
- [9. Density Modelling](#)
- [10. Conservation Law Diagnostics](#)
- [11. Simulation Units & Physical Constants](#)

1. Foundational Physics: The N-Body Problem

The simulation is built on the classical gravitational N-body problem. Given N point masses, the gravitational force exerted on body i by body j is Newton's law of universal gravitation:

$$F_{ij} = G * m_i * m_j / r_{ij}^2 \quad (\text{magnitude})$$

In vector form (direction from i toward j):

$$\vec{F}_{ij} = G * m_i * m_j * (\vec{r}_j - \vec{r}_i) / |\vec{r}_j - \vec{r}_i|^3$$

The resulting acceleration on body i is:

$$\vec{a}_i = \sum_{j \neq i} G * m_j * (\vec{r}_j - \vec{r}_i) / |\vec{r}_j - \vec{r}_i|^3$$

This sum must be evaluated for **all pairs**, giving $O(N^2)$ complexity for a naive implementation. Calculating forces for 10,000 bodies would require ~50 million pair evaluations per step. The engine solves this with the Barnes-Hut approximation described in Section 5.

Why N-Body?

Real astrophysical systems involve continuous mass distributions, but discretizing them as point masses (or extended spheres) captures essential physics:

- Orbital mechanics of planetary systems
- Galaxy formation and cluster dynamics

- Binary stars and tidal disruption events
- Star collisions and black hole accretion

2. Gravitational Softening

The Problem

The gravitational force **diverges** as two bodies approach each other ($r \rightarrow 0$), causing numerical instabilities and non-physical infinite accelerations.

The Solution: Softened Potential

The distance in the denominator is replaced with a **softened effective distance**:

$$r_{\text{eff}} = \sqrt{r^2 + \epsilon^2}$$

The softened gravitational force becomes:

$$F_{\text{softened}} = G * m_i * m_j / (r^2 + \epsilon^2)$$

which is equivalent to using the Plummer sphere potential:

$$\Phi_{\text{Plummer}}(r) = -G * m / \sqrt{r^2 + \epsilon^2}$$

This regularizes the potential so it remains finite at $r = 0$, giving a maximum force of $G * m_i * m_j / \epsilon^2$ regardless of how close bodies come.

Three Softening Modes in the Engine

Mode	Formula	Usage
Global	Single ϵ for all pairs	Simple simulations; uniform medium
PerBodyType	$\epsilon = (\epsilon_i + \epsilon_j) / 2$ per pair	Galaxy simulations mixing star/dark matter types
Adaptive	$\epsilon_i = \text{scale} * \sqrt[3]{m_i}$	Adaptive; heavier bodies have larger softening

The adaptive mode follows the inter-particle spacing scaling where softening is proportional to the cube-root of mass (analogous to the mean inter-particle distance for a uniform density cloud).

Typical value: $\epsilon = 1e-4$ (in simulation units, ~100 km for solar-mass bodies at 1 AU scale).

3. Numerical Integration: Moving Bodies Through Time

Integration is the process of numerically solving the equations of motion. The fundamental challenge is that gravity is nonlinear — forces change as positions change, so we must approximate the continuous trajectory with discrete steps.

3.1 Euler Integrator (Reference / Educational)

The simplest integrator. Given acceleration a at time t :

```
v(t + dt) = v(t) + a(t) * dt
x(t + dt) = x(t) + v(t) * dt
```

Why it fails for long simulations: Euler is **first-order** ($O(dt)$ error per step) and is **not symplectic** — it does not conserve a modified Hamiltonian. Energy drifts secularly (grows without bound over time), causing orbits to spiral outward or collapse artificially. Only used here for reference/comparison.

3.2 Velocity Verlet (Leapfrog KDK) — Primary Integrator

The Velocity Verlet is a **second-order symplectic** integrator using the Kick-Drift-Kick (KDK) sequence. Symplectic means it preserves the volume of phase space exactly, which leads to bounded (oscillating, not drifting) energy error over long simulations.

Algorithm (one complete step of size `dt`):

```
Step 1 – Half-Kick (½ velocity update using OLD acceleration):
    v(t + dt/2) = v(t) + a(t) * dt/2

Step 2 – Drift (position advance using half-updated velocity):
    x(t + dt) = x(t) + v(t + dt/2) * dt

Step 3 – Force Evaluation (compute new forces at new positions):
    a(t + dt) = F(x(t + dt)) / m

Step 4 – Half-Kick (½ velocity update using NEW acceleration):
    v(t + dt) = v(t + dt/2) + a(t + dt) * dt/2
```

Why this is superior to Euler:

- **Second-order accuracy:** $O(dt^2)$ global error vs Euler's $O(dt)$
- **Symplectic structure:** energy oscillates around the true value but never drifts
- **Time-reversible:** running the integrator backward with `-dt` recovers identical results
- **Low memory:** only requires storing positions, velocities, and the previous step's accelerations

This is the industry-standard integrator for long-duration N-body simulations (used in real galaxy-formation codes like GADGET and Arepo).

Code location: `VerletIntegrator.cs` and `SoAVerletIntegrator.cs` (optimized Structure-of-Arrays variant)

3.3 Runge-Kutta 4th Order (RK4)

RK4 evaluates the derivative (acceleration) at **four sub-points** within the timestep and combines them using weighted averaging:

```
k1 = f(t, x, v)
k2 = f(t + dt/2, x + k1*dt/2, v + k1_acc*dt/2)
k3 = f(t + dt/2, x + k2*dt/2, v + k2_acc*dt/2)
k4 = f(t + dt, x + k3*dt, v + k3_acc*dt)

x(t+dt) = x(t) + dt/6 * (k1_vel + 2*k2_vel + 2*k3_vel + k4_vel)
v(t+dt) = v(t) + dt/6 * (k1_acc + 2*k2_acc + 2*k3_acc + k4_acc)
```

Properties:

- **Fourth-order accuracy:** $O(dt^4)$ global error (much more accurate per step than Verlet)
- **NOT symplectic:** exhibits **secular energy drift** over very long timescales
- **4× more force evaluations** per step than Leapfrog (expensive for N-body)

When to use: Short-term trajectory prediction where absolute accuracy per step matters more than long-term energy conservation (e.g., spacecraft trajectories, spacecraft rendezvous simulations).

Code location: `RK4Integrator.cs`

3.4 Yoshida 4th-Order Symplectic — Native Engine

The Yoshida integrator achieves **4th-order symplectic** integration by composing THREE leapfrog substeps with specially tuned Forest-Ruth coefficients:

$$w_1 = 1 / (2 - \sqrt[3]{2}) \approx 1.3512071919596578$$

$$w_0 = -\sqrt[3]{2} / (2 - \sqrt[3]{2}) \approx -1.7024143839193153$$

Verify: $2 \cdot w_1 + w_0 = 1.0$ (total timestep preserved)

Full step of size `dt` :

Substep 1: KDK leapfrog with $dt_1 = w_1 \cdot dt$
 Substep 2: KDK leapfrog with $dt_2 = w_0 \cdot dt$ ← NEGATIVE! Runs backward in time
 Substep 3: KDK leapfrog with $dt_3 = w_1 \cdot dt$

The **negative middle step** ($w_0 \approx -1.70$) is mathematically intentional — it produces a 4th-order cancellation of error terms. The simulation briefly runs backward in time during each step, which is physically valid in time-reversible Hamiltonian mechanics.

Properties:

- **4th-order symplectic:** $O(dt^4)$ error while maintaining symplectic structure (bounded energy)
- Requires 3× the force evaluations of a single Leapfrog step
- Ideal for long-duration, high-accuracy simulations (planetary systems, binary stars)
- **GPU-optimized:** in GPU Barnes-Hut mode, all three substeps run entirely on device with a single upload/download

Code location: `engine/src/sim/engine.cpp` → `step_yoshida4_*` variants,
`engine/include/celestial/sim/yoshida.hpp`

4. Adaptive Timestep Control

For systems with widely varying accelerations (e.g., a tight binary near a supermassive black hole), a **fixed** timestep wastes computation on slow regions and misses fast dynamics in dense regions.

Adaptive DT Formula

After each step, the timestep is updated based on the **maximum acceleration** in the system:

$$dt_{next} = \eta \cdot \sqrt{\epsilon / a_{max}}$$

Where:

- `η` = safety factor (default 0.01) — controls conservatism

- ϵ = softening length (prevents divergence as $a_{\max} \rightarrow \infty$)
- $a_{\max}$ = maximum acceleration magnitude across all active bodies

This formula is derived from requiring that the displacement from one step is bounded: the "timescale" of the fastest-moving body is $\sim \sqrt{\epsilon / a_{\max}}$, which is the time to travel a softening length under maximum acceleration.

Clamping

$$dt_{\min} \leq dt_{\text{next}} \leq dt_{\max}$$

Prevents the timestep from growing too large (missing dynamics) or shrinking too small (hanging the simulation during extreme close encounters).

Note: Adaptive timestep is **disabled in deterministic mode** because varying dt across runs breaks bit-exact reproducibility.

5. Barnes-Hut $O(N \log N)$ Gravity Approximation

The Core Idea

Distant groups of bodies can be approximated as a **single point mass** at their center-of-mass without significant accuracy loss. This observation, formalized by Barnes and Hut (1986), reduces computation from $O(N^2)$ to $O(N \log N)$.

Octree Construction

The simulation volume is recursively subdivided into an **octree** (3D tree with 8 children per node):

1. Start with a root cell containing all N bodies
2. If a cell contains more than 1 body, subdivide it into 8 equal octants
3. Recursively subdivide until each leaf contains exactly 1 body
4. At each internal node, compute the **center-of-mass** and **total mass** of all contained bodies

This tree is rebuilt every step because bodies move.

The Opening Criterion

When computing the force on body i , traverse the octree. For each node:

If $(2 \cdot s)^2 < \theta^2 \cdot d^2 \rightarrow$ Use monopole approximation (node treated as point mass)
 Otherwise $\rightarrow$ Recurse into node's children

Where:

- s = node side length (spatial size of the cell)
- d = distance from body i to the node's center-of-mass
- θ (theta) = the **opening angle parameter** (default 0.5)

This criterion ensures that only nodes "far enough" relative to their size are approximated. Small $\theta \rightarrow$ fewer approximations $\rightarrow$ more accurate but slower. Large $\theta \rightarrow$ more approximations $\rightarrow$ faster but less accurate.

θ value	Accuracy	Speed
0.0	Exact ($O(N^2)$ equivalent)	Slowest

0.2	Very high (~0.01% error)	Slow
0.5	Standard (~0.1% error)	Fast
1.0	Low (~1% error)	Very fast

Theta floor: The engine enforces `θ_min = 0.2` to prevent degenerate behavior where almost every node forces direct computation.

Monopole vs. Multipole

The current implementation uses **monopole-only** (zeroth-order) approximation: each tree node is approximated as a single point mass at its center-of-mass. This is the standard Barnes-Hut approximation.

A placeholder interface `IMultipoleExpansion` exists for future **multipole** (higher-order) approximations that would include quadrupole terms, improving accuracy for a given θ without increasing computation.

Collision Detection During Traversal

A key optimization: in CPU Barnes-Hut and GPU Barnes-Hut modes, collision detection is **"free"** — it happens at the leaf nodes during the same force traversal. When the traversal reaches a leaf and the distance between body i and the leaf body j satisfies `dist < r_i + r_j`, a collision is flagged. No separate $O(N^2)$ collision pass is needed.

6. Collision Physics

The collision system operates as a **3-stage pipeline**: Detection → Resolution → Compaction.

6.1 Detection

Bodies are modeled as **solid spheres** with physical radii. A collision between body i and body j occurs when:

$$|r_i - r_j| < r_i + r_j$$

i.e., when the sphere surfaces overlap. The detection method depends on the compute mode:

- **CPU Brute-Force:** $O(N^2)$ pairwise scan after each step
- **CPU Barnes-Hut:** Free detection at leaf nodes during force traversal ($O(N \log N)$)
- **GPU Brute-Force:** CPU brute-force scan after GPU download
- **GPU Barnes-Hut:** GPU tree traversal with GPU-side atomic collision pair detection

6.2 Elastic Collisions

Elastic collisions conserve both **kinetic energy** and **momentum** (coefficient of restitution $e = 1.0$).

For two bodies colliding along contact normal $\hat{n}$:

$$j = -(1 + e) * (v_{rel} \cdot \hat{n}) / (1/m_i + 1/m_j)$$

$$v_{i_new} = v_i + (j / m_i) * \hat{n}$$

$$v_{j_new} = v_j - (j / m_j) * \hat{n}$$

Where $v_{rel} = v_i - v_j$ is the relative velocity and j is the impulse magnitude.

6.3 Inelastic Collisions

Inelastic collisions conserve momentum but **dissipate kinetic energy** via the coefficient of restitution $0 \leq e \leq 1$:

- $e = 1.0$: perfectly elastic (kinetic energy conserved)
- $e = 0.0$: perfectly inelastic (bodies stick together after collision; same as merge)
- $0 < e < 1$: partially inelastic (real-world collisions)

The same impulse formula applies with the chosen e value. At $e = 0$, the bodies have the same final velocity (center-of-mass velocity), equivalent to a merge.

6.4 Merge / Accretion — Conservation Laws

When bodies undergo **gravitational merging** (accretion), the following conservation laws are strictly enforced:

Mass Conservation (absolute — no mass is lost):

$$m_{total} = m_A + m_B$$

Momentum Conservation (linear):

$$v_{merged} = (m_A * v_A + m_B * v_B) / m_{total}$$

Position (Center of Mass):

$$r_{merged} = (m_A * r_A + m_B * r_B) / m_{total}$$

Energy: Kinetic energy is **NOT explicitly conserved** — the loss is physically valid, representing energy radiated as heat, sound, and light during the violent merger event.

Density (mass-weighted average):

$$\rho_{merged} = (m_A * \rho_A + m_B * \rho_B) / m_{total}$$

Radius recomputation (density-preserving merge):

$$r_{merged} = \sqrt[3]{(3 * m_{total} / (4\pi * \rho_{merged}))}$$

Body Type Promotion Rules

When bodies of different types merge, the engine applies physical promotion rules:

Merger	Outcome
Any + Black Hole	Black Hole (always wins — absorbs everything)
Star + Star (high mass)	Neutron Star or Black Hole (depending on combined mass threshold)
Planet + Planet	Larger Planet
Any + Neutron Star	Neutron Star (unless combined mass exceeds Tolman-Oppenheimer-Volkoff limit)

Merge Safeguards

To prevent cascade collapse (one massive body merging with everything in a single step):

- 64 merges per frame** hard cap

- **2 merges per body per frame** hard cap

These limits prevent catastrophic numerical instability during dense cluster collapses.

7. Post-Newtonian Relativistic Corrections (1PN)

Motivation

Newton's law of gravity is only approximately correct. In the presence of strong gravity or high velocities, **General Relativity** must be used. The 1PN (First Post-Newtonian) corrections add relativistic effects to the Newtonian equations of motion as a perturbative expansion in powers of v^2/c^2 and $GM/(rc^2)$.

These corrections capture key relativistic effects such as:

- Orbital precession (e.g., the famous 43 arcsecond/century precession of Mercury)
- Coupling between position and velocity in the gravitational field
- Velocity-dependent drag-like terms from the gravitational "frame-dragging" in weak fields

The Einstein-Infeld-Hoffmann 1PN Formula

The additional acceleration on body **i** due to body **j** is:

$$\begin{aligned} \mathbf{a}_{i,1PN} = & (G \cdot m_j / (r^2 \cdot c^2)) \times \{ \\ & \hat{n} \cdot [-v_i^2 - 2 \cdot v_j^2 + 4 \cdot (v_i \cdot v_j) + 1.5 \cdot (\hat{n} \cdot v_j)^2 \\ & \quad + 5 \cdot G \cdot m_i / r + 4 \cdot G \cdot m_j / r] \\ & + (v_i - v_j) \cdot [4 \cdot (\hat{n} \cdot v_i) - 3 \cdot (\hat{n} \cdot v_j)] \\ & \} \end{aligned}$$

Where:

- $\hat{n} = (\mathbf{r}_{ij} - r_{ij}) / |\mathbf{r}_{ij} - r_{ij}|$ — unit vector from i to j
- $r = |\mathbf{r}_{ij} - r_{ij}|$ — separation distance (softened by ϵ)
- $\mathbf{v}_i, \mathbf{v}_j$ — velocity vectors of bodies i and j
- c — speed of light in simulation units

Term 1 (scalar multiplied by $\hat{n}$): Accounts for the change in effective gravitational mass due to kinetic and potential energy of both bodies. The $+5 \cdot G \cdot m_i / r + 4 \cdot G \cdot m_j / r$ terms represent how gravity itself is affected by the gravitational potential energy already present.

Term 2 (along relative velocity): Velocity-dependent corrections representing the finite speed of gravity and coupling between orbital angular momentum and gravitational field.

Safety Conditions

The 1PN expansion is a **weak-field, slow-motion approximation** and breaks down in strong gravity:

Condition	Action
$v_{rel} > 0.3c$	Skip correction — PN expansion invalid at relativistic velocities
$r < 3 \cdot R_{Schwarzschild}$	Skip correction — strong-field regime requires full GR

Schwarzschild Radius Warning

The engine computes the combined Schwarzschild radius:

$$R_s = 2 \cdot G \cdot (m_i + m_j) / c^2$$

When a pair approaches within $3 \cdot R_s$, a proximity warning is issued and the 1PN correction is skipped. Full General Relativistic treatment would be required at these scales.

References:

- Blanchet, *Living Reviews in Relativity* 17 (2014)
- Poisson & Will, *Gravity* (Cambridge, 2014), §9.3

8. Gravitational Wave Physics

What Are Gravitational Waves?

According to General Relativity, accelerating masses produce ripples in spacetime called **gravitational waves**. These propagate at the speed of light and carry energy away from the system.

The engine implements the **quadrupole formula** (lowest-order GW emission), which is valid for non-relativistic, non-extreme-mass-ratio systems.

Gravitational Wave Strain

The dimensionless strain h (the fractional change in distance caused by the wave) for a binary system is:

$$h \approx 4 \cdot G^2 \cdot m_1 \cdot m_2 / (c^4 \cdot r \cdot d)$$

Where:

- m_1, m_2 — masses of the two bodies (in solar masses)
- r — orbital separation
- d — observer distance from the binary system
- h scales as $1/d$ (inverse-square for energy, inverse-linear for amplitude)

The prefactor $4 \cdot G^2 / c^4$ makes h extremely small — for LIGO's detected mergers, $h \sim 10^{-21}$, meaning spacetime is stretched by 1 part in 10^{21} .

Energy Loss Rate (GW Backreaction)

Gravitational wave emission drains orbital energy, causing the two bodies to spiral inward (the classic "chirp" of a compact binary inspiral):

$$dE/dt = -(32/5) \cdot G^4 \cdot (m_1 \cdot m_2)^2 \cdot (m_1 + m_2) / (c^5 \cdot r^5)$$

The r^{-5} dependence means:

- GW emission is negligible at wide separations
- Emission grows rapidly as the binary tightens
- This is the physical mechanism that drove GW150914 to merger in under a billion years

Implementation

Each timestep, all binary pairs are sampled:

1. Compute strain h and energy loss dE/dt for each pair
2. Sum total strain over all pairs

3. Record strain vs. time into the **WaveformBuffer** (ring buffer of size 8192)
4. Accumulate total radiated energy: `E_radiated += |dE/dt| * dt`

In simulation units ($G = 1$), the prefactors reduce to:

```
StrainPrefactor      = 4 / c^4_sim
EnergyLossPrefactor  = -32 / (5 * c^5_sim)
```

9. Density Modelling

Each body is assigned a **physical density** ρ (kg/m^3 in SI; in simulation units for N-body). The radius is derived from mass and density using the sphere volume formula:

```

$$\rho = m / V = m / (4\pi/3 \cdot r^3)$$
  
  
Solving for r:  

$$r = \sqrt[3]{3 \cdot m / (4\pi \cdot \rho)}$$

```

Default density: `5515 kg/m³` — the mean density of the Earth, a reasonable baseline for rocky bodies.

Density-Preserving Merge

After a merge event, the new radius is computed from the **merged mass and merged density**:

```

$$\rho_{\text{merged}} = (m_A \cdot \rho_A + m_B \cdot \rho_B) / (m_A + m_B) \quad [\text{mass-weighted average}]$$
  

$$r_{\text{merged}} = \sqrt[3]{3 \cdot m_{\text{merged}} / (4\pi \cdot \rho_{\text{merged}})}$$

```

This is physically more accurate than volume conservation ($r_{\text{merged}} = \sqrt[3]{r_A^3 + r_B^3}$) because real bodies compress under gravity — a merged star is denser than the sum of its parts until nuclear pressure stabilizes it.

Black Hole Special Case

When a body becomes a **Black Hole** (by type promotion or formation), its radius is set to its **Schwarzschild radius**:

```

$$R_s = 2 \cdot G \cdot M / c^2$$

```

This is the event horizon — the boundary beyond which nothing can escape. In simulation units ($G = 1$), this simplifies to `R_s = 2 * M / c^2_sim`.

10. Conservation Law Diagnostics

Monitoring conservation of physical quantities is the gold standard for validating N-body simulations. The engine tracks six conserved quantities:

Quantity	Formula	Ideal Drift
Total Energy	$E = KE + PE$	Oscillates, never drifts (symplectic)
Linear Momentum	$\mathbf{p} = \sum m_i \mathbf{v}_i$	0 (Galilean invariance)
Angular Momentum	$\mathbf{L} = \sum m_i (\mathbf{r}_i \times \mathbf{v}_i)$	0 (rotational invariance)

Center of Mass	$\mathbf{r}_{\text{COM}} = \sum m_i \mathbf{r}_i / M$	Moves at constant velocity (Galilean)
Total Mass	$M = \sum m_i$	0 (mass conservation)
Virial Ratio	$Q = 2 \cdot \text{KE} /$	PE

Kinetic Energy

$$\text{KE} = \sum_i \frac{1}{2} \cdot m_i \cdot |\mathbf{v}_i|^2$$

Potential Energy (Pairwise)

$$\text{PE} = \sum_i \sum_{j>i} -G \cdot m_i \cdot m_j / \text{sqrt}(r_{ij}^2 + \epsilon^2)$$

The softened denominator ensures PE is finite even at close separations. For $N > 256$ in Barnes-Hut mode, PE is computed via tree traversal ($O(N \log N)$) instead of $O(N^2)$ pairwise summation.

Energy Drift Metric

The relative energy drift tracks the accuracy of the simulation over time:

$$\Delta E/E_0 = |E(t) - E_0| / |E_0|$$

Where E_0 is the energy at the first snapshot. **Thresholds:**

- $\Delta E/E_0 < 0.1\%$ — excellent (default pass threshold)
- $\Delta E/E_0 < 1.0\%$ — acceptable for long simulations
- $\Delta E/E_0 > 5.0\%$ — indicates instability (timestep too large, or close encounter)

Symplectic integrators (Verlet, Yoshida) produce energy that **oscillates** around the true value — the drift metric stays bounded perpetually. **Non-symplectic integrators** (RK4, Euler) produce energy that **drifts** monotonically, eventually failing the diagnostic.

Rolling Averages

All diagnostics are computed over a **300-sample rolling window** to suppress per-step fluctuations and report long-term trends.

11. Simulation Units & Physical Constants

The engine operates in **N-body units** to avoid floating-point precision loss from extremely large or small SI values:

Quantity	Simulation Unit	Approximate SI Equivalent
Mass	1 M $\odot$ (solar mass)	1.989×10^{30} kg
Length	1 AU (astronomical unit)	1.496×10^{11} m
Time	1 yr / (2 π)	~58.13 days
G (gravity)	G_sim = 1	6.674×10^{-11} N·m ² /kg ²
c (light)	~63,196 AU/yr	2.998×10^8 m/s

Why G = 1?

Normalizing $G = 1$ is a standard technique in N-body astrophysics. In these units:

- Orbital period of $1 M_{\odot} + 1 \text{ AU}$ orbit = 2π time units (by Kepler's Third Law)
- All force calculations reduce to $\sum m_j \cdot \mathbf{r}_{ij} / |\mathbf{r}_{ij}|^3$
- Double-precision floating point can represent the relevant mass ratios (10^{-6} to 10^9) without loss

Schwarzschild Radius in Simulation Units

$$R_s = 2 \cdot G \cdot M / c^2 \rightarrow (G=1) \rightarrow R_s = 2 \cdot M / c^2_{\text{sim}}$$

For a $1 M_{\odot}$ black hole: $R_s \approx 9.87 \times 10^{-9} \text{ AU} \approx 1476 \text{ m}$

This document covers the complete physics model of the Celestial Mechanics engine as of March 2026. All formulas implemented in source code have been verified against published literature references.